

Project Blog

Project updates and... um...

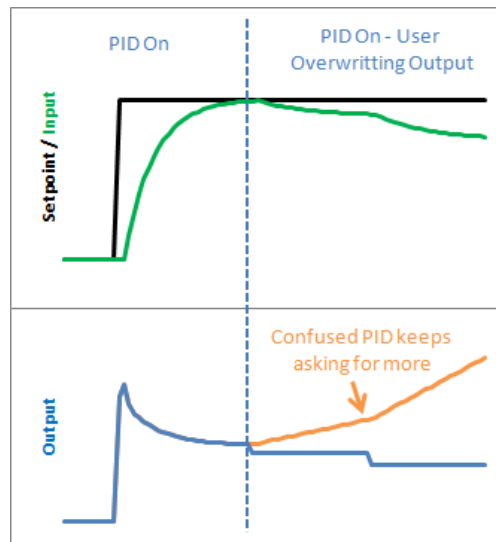
Improving the Beginner's PID: On/Off

Posted on April 15, 2011 by Brett

(This is Modification #5 in a [larger series](#) on writing a solid PID algorithm)

The Problem

As nice as it is to have a PID controller, sometimes you don't care what it has to say.



Let's say at some point in your program you want to force the output to a certain value (0 for example) you could certainly do this in the calling routine:

```
void loop()
{
  Compute();
  Output=0;
}
```

This way, no matter what the PID says, you just overwrite its value. This is a terrible idea in practice however. The PID will become very confused: "I keep moving the output, and nothing's happening! What gives?! Let me move it some more." As a result, when you stop over-writing the output and switch back to the PID, you will likely get a huge and immediate change in the output value.

The Solution

The solution to this problem is to have a means to turn the PID off and on. The common terms for these states are "Manual" (I will adjust the value by hand) and "Automatic" (the PID will automatically adjust the output). Let's

see how this is done in code:

The Code

```

1  /*working variables*/
2  unsigned long lastTime;
3  double Input, Output, Setpoint;
4  double ITerm, lastInput;
5  double kp, ki, kd;
6  int SampleTime = 1000; //1 sec
7  double outMin, outMax;
8  bool inAuto = false;
9
10 #define MANUAL 0
11 #define AUTOMATIC 1
12
13 void Compute()
14 {
15     if(!inAuto) return;
16     unsigned long now = millis();
17     int timeChange = (now - lastTime);
18     if(timeChange>=SampleTime)
19     {
20         /*Compute all the working error variables*/
21         double error = Setpoint - Input;
22         ITerm+= (ki * error);
23         if(ITerm> outMax) ITerm= outMax;
24         else if(ITerm< outMin) ITerm= outMin;
25         double dInput = (Input - lastInput);
26
27         /*Compute PID Output*/
28         Output = kp * error + ITerm- kd * dInput;
29         if(Output > outMax) Output = outMax;
30         else if(Output < outMin) Output = outMin;
31
32         /*Remember some variables for next time*/
33         lastInput = Input;
34         lastTime = now;
35     }
36 }
37
38 void SetTunings(double Kp, double Ki, double Kd)
39 {
40     double SampleTimeInSec = ((double)SampleTime)/1000;
41     kp = Kp;
42     ki = Ki * SampleTimeInSec;
43     kd = Kd / SampleTimeInSec;
44 }
45
46 void SetSampleTime(int NewSampleTime)
47 {
48     if (NewSampleTime > 0)
49     {
50         double ratio = (double)NewSampleTime
51                        / (double)SampleTime;
52         ki *= ratio;
53         kd /= ratio;
54         SampleTime = (unsigned long)NewSampleTime;
55     }
56 }
57
58 void SetOutputLimits(double Min, double Max)

```

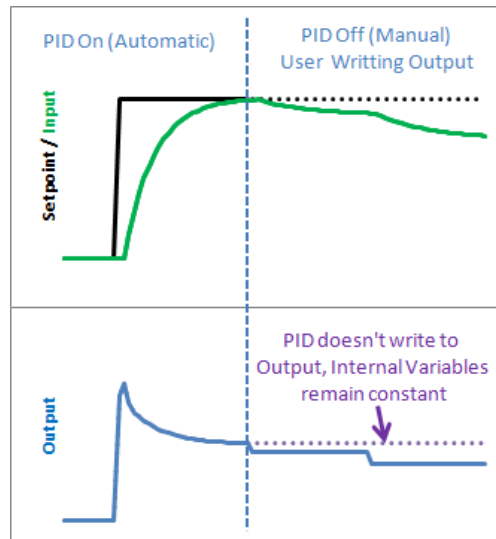
```

59 {
60     if(Min > Max) return;
61     outMin = Min;
62     outMax = Max;
63
64     if(Output > outMax) Output = outMax;
65     else if(Output < outMin) Output = outMin;
66
67     if(ITerm > outMax) ITerm = outMax;
68     else if(ITerm < outMin) ITerm = outMin;
69 }
70
71 void SetMode(int Mode)
72 {
73     inAuto = (Mode == AUTOMATIC);
74 }

```

A fairly simple solution. If you're not in automatic mode, immediately leave the Compute function without adjusting the Output or any internal variables.

The Result



It's true that you could achieve a similar effect by just not calling Compute from the calling routine, but this solution keeps the workings of the PID contained, which is kind of what we need. By keeping things internal we can keep track of which mode were in, and more importantly it let's us know when we change modes. That leads us to the next issue...

[Next >>](#)

 [Creative Commons License](#)

This entry was posted in [Coding](#), [PID](#) and tagged [Arduino](#), [Beginner's PID](#), [PID](#). Bookmark the [permalink](#).

10 Responses to *Improving the Beginner's PID: On/Off*



Autopilot says:

July 26, 2011 at 6:15 pm

Hello,

Thanks for your tutorial but isn't it a good idea to reset integral sum before resuming automatic control after manual mode? Let's imagine controller had accumulated some output in integral term, then it's switched to manual mode for some time, then again back to automatic. In that case controller would try to use old accumulated value (based on history) under perhaps different conditions. What's your view?



Brett says:

July 27, 2011 at 6:34 am

Definitely agree on that. That's covered in the next section: Initialization.



Adrian says:

August 19, 2011 at 8:35 am

I believe there is an error in the definition of the timeChange variable, on line 17. I have found that it is possible that timeChange can become negative when switching between MANUAL and AUTOMATIC. This was stopping the subsequent if statement from return to correct value, resulting in the PID not updating.

Changing the definition from int to unsigned long appears to have resolved this issue for me.

Oh, and thanks a heap for both the Arduino PID library, and your excellent documentation.

Best regards,

Adrian



Brett says:

August 19, 2011 at 8:50 am

@Adreian

yes that is indeed an error. there's a conversation on the [sample time](#) page that offers a solution.



Adrian says:

August 19, 2011 at 9:20 am

Thanks for the fast response confirming my hack solution might actually work. :)

Cheers, Adrian



Kyle says:

April 9, 2012 at 1:25 pm

One 'complaint', line 15, `if(!inAuto) return;`, is bad programming. A more correct way would be `while(inAuto) { ... }`. Also, I was trying to think of a way to fix your last change that was written the same way, `if(Min > Max) return;`. You could rewrite it the same way. Both changes would require you to make the test false at the end of the while loop, so it would drop out properly. Yes, it is more code, but, it is written more correctly.



Kyle says:

 April 9, 2012 at 11:32 pm

Stupid me. I should have suggested you just change your logic instead of changing the language completely. So, rather than `if(!inAuto) return;` you should do `if (inAuto) { ... }`. The same goes for the `if (Min > Max) return;`. It should be `if (Min < Max) { ... }`. The preceding follows good programming practices.

**Anthony says:**

April 30, 2012 at 1:32 pm

Great article!

I just wanted to respond to Kyle's comments about the "bad" programming style of having multiple returns in your function. There's nothing wrong with that. In fact, if multiple conditions decide whether to exit a function, it's sure as hell a whole lot nicer to have a return there instead of a large amount of nested ifs. In this case, the return at line 15 makes it very clear what will happen in manual mode, so I prefer it to the if clause around the whole rest of the function (as Kyle suggests). Just my 2 cents.

Cheers, Anthony

**Terry G says:**

April 14, 2016 at 8:25 am

I think there is a error with the integral term of the PID controller. When used in conjunction with the kp term the resulting PI controller eliminates the error as desired. However when the integral term is set to zero the output remains constant, that is the output does not revert back to the error one would expect when using kp alone. Could be wrong but this is not usual PID action.

**Brett says:**

April 14, 2016 at 8:36 am

@Terry G, "usual PID action" varies on this point. depending on the manufacturer, you can set a bias (or baseline) value from which the pid can operate, or the p-only controller can always work from 0. I decided to choose the first option, since I have encountered various processes where p-only is fine, but you want the output to be centered around 70% say. also, if you want this algorithm to operate from a baseline of 0, it's simply a matter of setting output=0 before putting the pid in automatic. everybody's happy!

Project Blog

Proudly powered by [WordPress](#).